Day 2: Advanced Array Algorithms & Matrix Manipulation

Goal: Master 1D algorithmic techniques (Two-Pointers, Sliding Window, Binary Search) and complex 2D matrix traversals.

Total Estimated Time: ~7 Hours.

Block 1: 1D Array Algorithmic Patterns (2 Hours)

1. The Dutch National Flag Problem (30 mins)

- **Task:** You are given an array containing only 0s, 1s, and 2s (e.g., [2, 0, 2, 1, 1, 0]). Sort the array in-place so that objects of the same color are adjacent, in the order 0, 1, and 2.
- **Constraint:** Do this in a single pass $O(n)$ time complexity and $O(1)$ constant space.

2. Kadane's Algorithm: Maximum Subarray Sum (30 mins)

- **Task:** Given an integer array (which may contain negative numbers), find the contiguous subarray (containing at least one number) which has the largest sum and return its sum. Example: [-2, 1, -3, 4, -1, 2, 1, -5, 4] should return 6 (from subarray [4, -1, 2, 1]).
- **Constraint:** $O(n)$ time complexity.

3. Array Rotation (In-Place) (30 mins)

- **Task:** Given an array, rotate the array to the right by k steps, where k is non-negative. Example: Array [1, 2, 3, 4, 5, 6, 7] and $k = 3$ becomes [5, 6, 7, 1, 2, 3, 4].
- **Constraint:** Do it in $O(1)$ extra space. (Hint: Think about reversing sub-sections of the array).

4. Trapping Rain Water (30 mins)

- **Task:** Given n non-negative integers representing an elevation map where the width of each bar is 1, compute how much water it can trap after raining. Example: [0, 1, 0, 2, 1, 0, 1, 3, 2, 1, 2, 1] traps 6 units of water.

Block 2: Searching and Optimization (2 Hours)

5. Binary Search on a Rotated Sorted Array (40 mins)

- **Task:** There is an integer array sorted in ascending order, but it is rotated at an unknown pivot index (e.g., [0, 1, 2, 4, 5, 6, 7] might become [4, 5, 6, 7, 0, 1, 2]). Given a target value, return its index if found, otherwise return -1.
- **Constraint:** You must write an algorithm with $O(\log n)$ runtime complexity.

6. The "3Sum" Problem (40 mins)

- **Task:** Given an integer array, find all unique triplets in the array which give the sum of

zero. Example: [-1, 0, 1, 2, -1, -4] should return [[-1, -1, 2], [-1, 0, 1]].
- **Constraint:** The solution set must not contain duplicate triplets.

7. Sliding Window: Maximum Sum of Size K (40 mins)

- **Task:** Given an array of integers and a number k, find the maximum sum of a contiguous subarray of size k.
- **Focus:** Instead of recalculating the sum of k elements every time, subtract the element leaving the window and add the element entering the window.

Block 3: 2D Arrays and Matrices (1.5 Hours)

8. Spiral Matrix Traversal (30 mins)

- **Task:** Given an $m \times n$ matrix, return all elements of the matrix in spiral order.
  Example:
  Plaintext
  Input:
  [1, 2, 3]
  [4, 5, 6]
  [7, 8, 9]
  Output: [1, 2, 3, 6, 9, 8, 7, 4, 5]

- **Focus:** Managing four boundary variables (top, bottom, left, right) and shrinking them as you traverse.

9. Rotate Matrix 90 Degrees In-Place (30 mins)

- **Task:** You are given an $n \times n$ 2D matrix representing an image. Rotate the image by 90 degrees (clockwise).
- **Constraint:** You must rotate the matrix in-place. Do not allocate another 2D matrix for the rotation. (Hint: Transpose the matrix first, then reverse each row).

10. Matrix Multiplication (30 mins)

- **Task:** Write a program to multiply two 2D arrays. If Matrix A is $m \times n$ and Matrix B is $n \times p$, the resulting Matrix C will be $m \times p$.
- **Formula:** $C_{i,j} = \sum_{k=1}^{n} A_{i,k} \times B_{k,j}$
- **Constraint:** Include a check to ensure the matrices can actually be multiplied (columns of A must equal rows of B). Throw an error message if they cannot.

Block 4: 2D Grid Simulations (1.5 Hours)

11. Minesweeper Board Generator (40 mins)

- **Task:** Given an $m \times n$ grid filled with characters where 'M' represents a mine and

'E' represents an empty square, calculate the adjacent mine counts for all 'E' squares (horizontally, vertically, and diagonally). Replace 'E' with the count (e.g., '1', '2').

- **Focus:** Boundary checking is critical here. You must avoid ArrayIndexOutOfBoundsException when checking the edges and corners of the board.

12. Conway's Game of Life (Next Generation) (50 mins)

- **Task:** Given an $m \times n$ grid representing the current state of Conway's Game of Life (0 = dead, 1 = alive), compute the next state based on the following rules:
  1. Any live cell with fewer than two live neighbors dies (underpopulation).
  2. Any live cell with two or three live neighbors lives.
  3. Any live cell with more than three live neighbors dies (overpopulation).
  4. Any dead cell with exactly three live neighbors becomes a live cell (reproduction).
- **Constraint:** All state changes must happen simultaneously. You cannot update the board in place cell-by-cell, because modifying a cell will immediately impact the neighbor count of the next cell you check.